Numerical Techniques in Hydrodynamics.

Tutorial - 5

By :-

J. Karthick

OE23S018

M.S. Scholar.

Dye Injection Flow

Problem statement:-

A dye injected in flow with $C_0 = 10 g/cm^3$. solvy the advection - diffusion problem to determine the nature of spread of the dye due to the velocities of 0, 1.0, 2.0 m/sec.

Aim:-

To determine the flow of the injection Dye based on the time period function using advection- diffusion problem.

Governing equations:-

$$\frac{\partial C}{\partial t} + u \frac{\partial C}{\partial x} = D \left(\frac{\partial^2 C}{\partial x^2} \right)$$

$C \rightarrow$ dye concentration (t, x) g/cm^3

$u \rightarrow$ flow velocity.

$D \rightarrow$ Diffusion co-efficient.

The above equation is 1D advection - Diffusion equation.

given data :-

1. Initial condition.

at $c = 10\ g/cm^3$

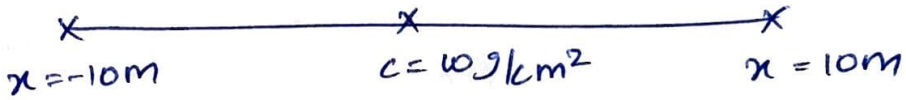
2. Diffusion co-efficient

$D = 0.5\ m^2/s$

3. Velocities.

0, 1, 2 m/sec.

4. Domain length



5. time steps. = 0.01

Discretization :-

This method is used to discretized by the Explicit method From the governing equation.

$$\frac{\partial c}{\partial t} + u\frac{\partial c}{\partial x} = D\frac{\partial^2 c}{\partial x^2}$$

↓ Time Derivative

↓ advection Term.

↓ Diffusion Term.

1. Time Derivative :-

$$\frac{\partial C}{\partial t} = \frac{C_i^{n+1} - C_i^n}{\Delta t}$$

2. advection term : (upwind difference).

$$u \frac{\partial C}{\partial x} = u \frac{C_i^n - C_{i-1}^n}{\Delta x}$$

3. Diffusion Term : central difference scheme.

$$D \frac{\partial^2 C}{\partial x^2} = D \left[\frac{C_{i+1}^n - 2C_i^n + C_{i-1}^n}{\Delta x^2} \right]$$

The above equations to substitute the governing the equation.

$$\frac{C_i^{n+1} - C_i^n}{\Delta t} + u \left[\frac{C_i^n - C_{i-1}^n}{\Delta x} \right] = D \left[\frac{C_{i+1}^n - 2C_i^n + C_{i-1}^n}{\Delta x^2} \right]$$

Solving this equation to the final solution will be,

$$C_i^{n+1} = C_i^n - \underbrace{\frac{u \Delta t}{\Delta x} [C_i^n - C_{i-1}^n]}_{\text{Advection Term.}} + \underbrace{\frac{D \Delta t}{\Delta x^2} [C_{i+1}^n - 2C_i^n + C_{i-1}^n]}_{\text{Diffusion Term.}}$$

Stability creteria :-

1. For advection :

Courant - Friedrich - Lewy Creteria.

$$CFL = \frac{u\,\Delta t}{\Delta x} \leq 1.$$

2. Diffusion Stability :-

Fourier Number,

$$= D\,\frac{\Delta t}{\Delta x^2} \leq 0.5$$

Numerical Data For Stability Calculation :-

$$\Delta x = 0.2$$

$L = -10$ to $10m \Rightarrow 20m$

$$\boxed{L = 20m}$$

$$Nx \Rightarrow \frac{L}{\Delta x} + 1 = \frac{20}{0.2} = 101 \text{ cells.}$$

$Nx \rightarrow$ Number of cells

$$\Delta t = 0.01$$

$$CFL = u\,\frac{\Delta t}{\Delta x} = 1 \times \frac{0.01}{0.2} \Rightarrow \boxed{0.05 \leq 1}$$

$$\boxed{\text{Stable}}$$

$$\text{Diffusivity} = D^2\,\frac{\Delta t}{\Delta x^2} \Rightarrow 0.5 \times \frac{0.01}{(0.2)^2}$$

$$\boxed{F = 0.0125} \leq 0.5$$

Stable.

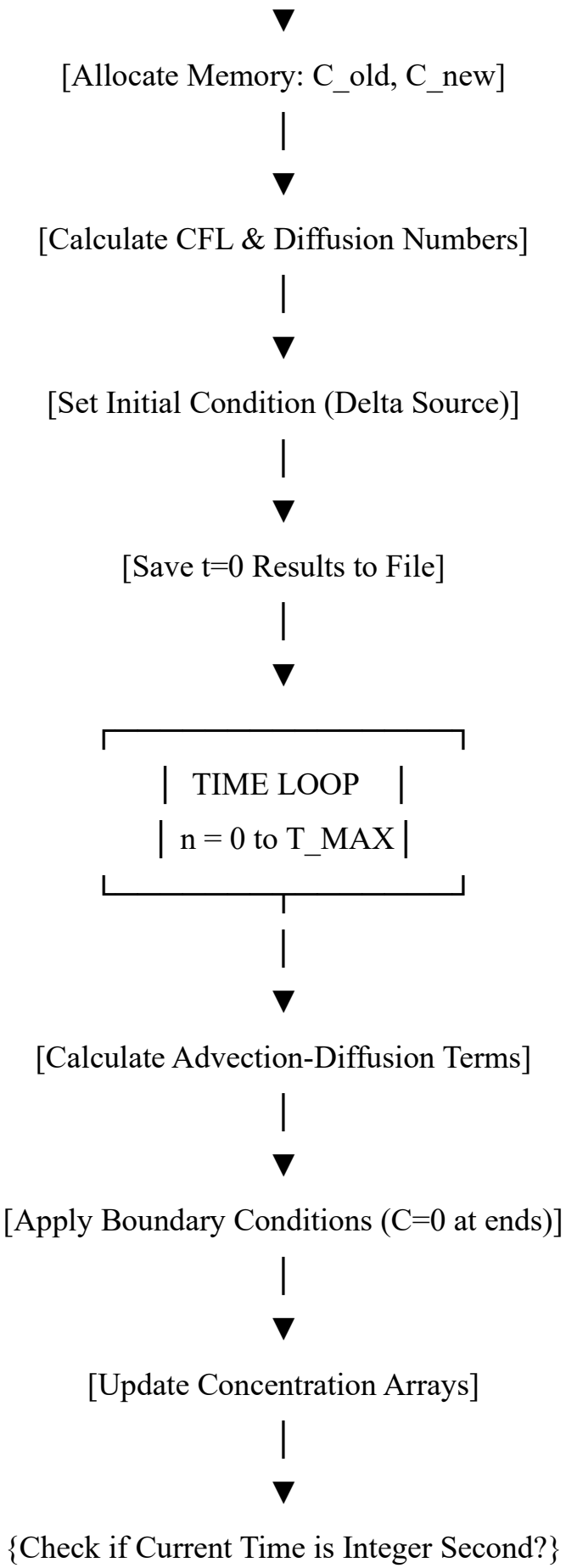
ALGORITHM:

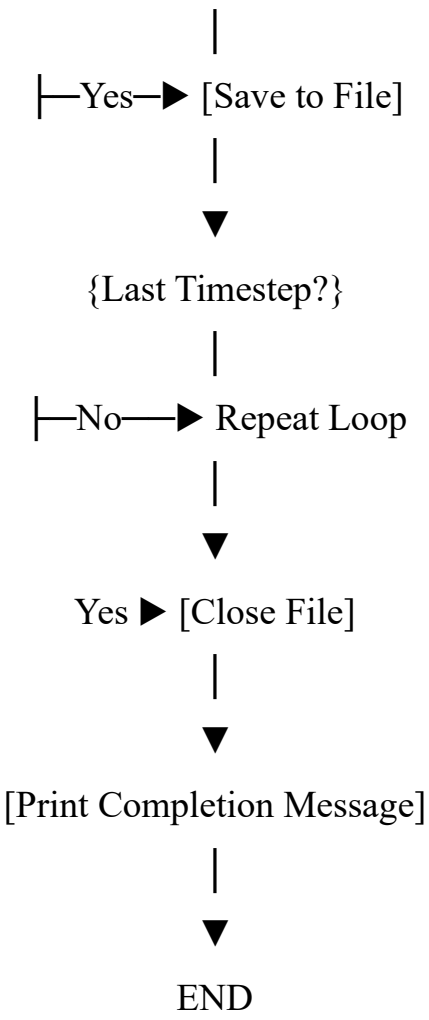
1. **Initialization Phase**
    - Define physical parameters (domain size, grid spacing, velocity, etc.)
    - Allocate memory for concentration arrays (C_old and C_new)
    - Calculate stability parameters (CFL and diffusion numbers)
    - Initialize concentration profile with a Dirac delta source at the center
    - Save initial condition (t=0) to file
2. **Time Integration Loop**
    i. For each timestep n = 0 to T_MAX/DT:
    b. Compute new concentrations:
        i. For each spatial point i (except boundaries):
            - Calculate advection term: $u*(C[i] - C[i-1])/\Delta x$
            - Calculate diffusion term: $D*(C[i+1] - 2C[i] + C[i-1])/\Delta x^2$
            - Update C_new[i] = C_old[i] - Δt*(advection) + Δt*(diffusion)
    c. Apply boundary conditions:
            - Set C_new[0] = 0 (left boundary)
            - Set C_new[NX-1] = 0 (right boundary)
    d. Copy C_new to C_old for next iteration
    e. If current_time is integer second:
        i. Save spatial concentration profile to file
3. **Termination Phase**
    - Close output file
    - Print completion message

FLOW CHART OF PROCESS:

START

[Initialize Parameters]


[Allocate Memory: C_old, C_new]
[Calculate CFL & Diffusion Numbers]
[Set Initial Condition (Delta Source)]
[Save t=0 Results to File]
TIME LOOP
n = 0 to T_MAX
[Calculate Advection-Diffusion Terms]
[Apply Boundary Conditions (C=0 at ends)]
[Update Concentration Arrays]
{Check if Current Time is Integer Second?}



C-LANGUAGE PROGRAM FOR THE PROBLEM:

```
#include <stdio.h>
#include <math.h>
//**********ADVECTION-DIFFUSION SIMULATION WITH CONTINUOUS SOURCE**********//
// Domain parameters
#define L 20.0        // Domain length (-10m to +10m)
#define DX 0.2        // Spatial step size (m)
#define Cd 10.0       // Initial concentration (kg/m³)
#define D 0.5         // Diffusion coefficient (m²/s)
#define DT 0.01       // Time step (s)
#define U 2.0         // Flow velocity (m/s)
#define T_MAX 100.0   // Total simulation time (s)
```

```
#define NX (int)(L/DX) + 1  // Number of spatial points
// Prototypes
void initialize(double C[], int center);
void update(double C[], double C_new[], double dx, double dt, double u, double
d, int center);
void save_results(FILE *fp, double C[], double time);
int main() {
    // Simulation parameters
    const int center = (NX-1)/2;
    const int num_steps = (int)(T_MAX/DT);
    double current_time = 0.0;

    // Concentration arrays
    double C_old[NX] = {0.0};
    double C_new[NX] = {0.0};

    // Stability checks
    const double CFL = U * DT / DX;
    const double diffusion_number = D * DT / (DX*DX);

    // File handling
    FILE *fp = fopen("results.txt", "w");
    if(!fp) {
        printf("Error opening output file!\n");
        return 1;
    }

    // Write simulation parameters
    fprintf(fp, "# Advection-Diffusion Simulation Parameters\n");
```

```
    fprintf(fp, "# Domain Length: %.1f m\n", L);
    fprintf(fp, "# Velocity: %.1f m/s\n", U);
    fprintf(fp, "# Diffusion Coefficient: %.1f m²/s\n", D);
    fprintf(fp, "# CFL Number: %.3f\n", CFL);
    fprintf(fp, "# Diffusion Number: %.3f\n\n", diffusion_number);
    // Initialization and save initial condition
    initialize(C_old, center);
    save_results(fp, C_old, 0.0);  // Save initial condition at t=0
    // Main simulation loop
    for(int n = 0; n < num_steps; n++) {
        current_time = (n + 1) * DT
        // Update concentrations
        update(C_old, C_new, DX, DT, U, D, center);

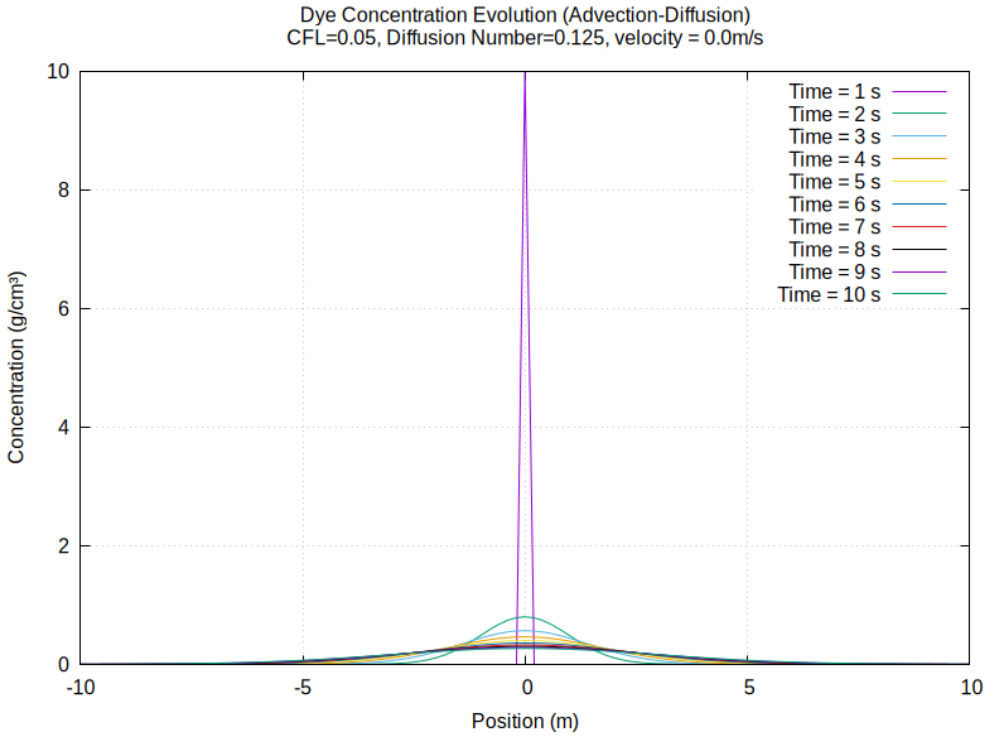
        // Save data at specified intervals
        if(fmod(current_time, 1.0) <= DT/2.0) {
            save_results(fp, C_old, current_time);
        }
    }
    fclose(fp);
    printf("Simulation completed successfully!\n");
    return 0;
}
void initialize(double C[], int center) {
    for(int i = 0; i < NX; i++) {
        C[i] = 0.0;
          // Continuous source at center
    }
   C[center] = Cd;
```

```
}

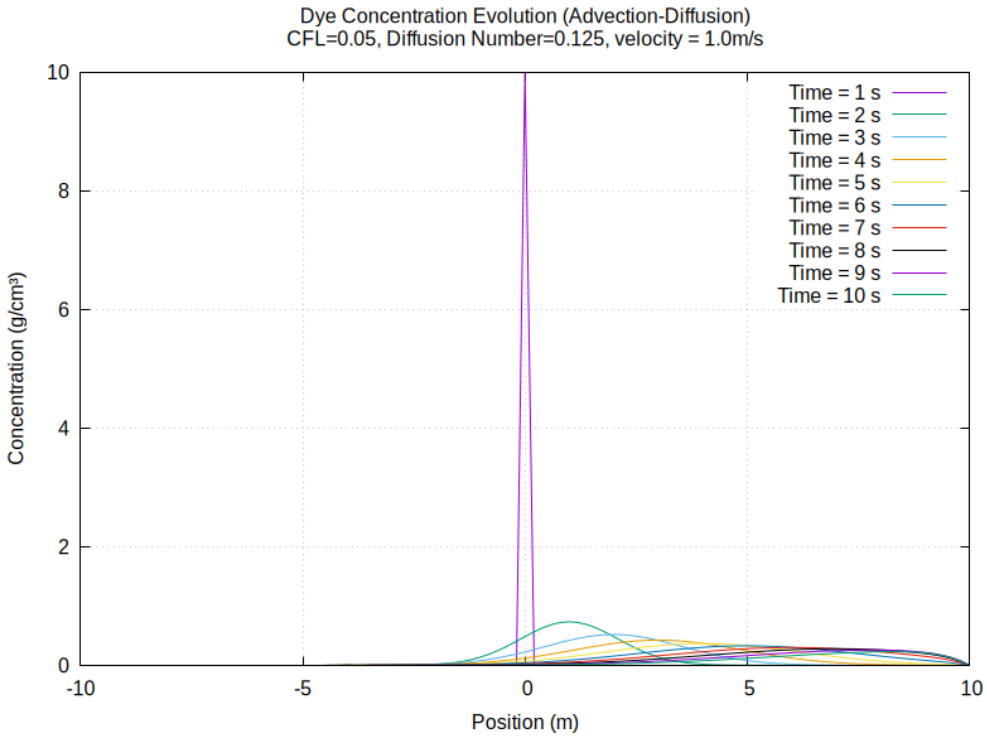

void update(double C[], double C_new[], double dx, double dt, double u, double
d, int center) {
    // Update interior points
    for(int i = 1; i < NX-1; i++) {
        const double adv = u * (C[i] - C[i-1]) / dx;
        const double diff = d * (C[i+1] - 2*C[i] + C[i-1]) / (dx*dx);
        C_new[i] = C[i] - dt * adv + dt * diff;
    }
    // Boundary conditions
    C_new[0] = 0.0;         // Upstream Dirichlet BC
    C_new[NX-1] = 0.0;      // Downstream Dirichlet BC
    // Update array
    for(int i = 0; i < NX; i++) {
        C[i] = C_new[i];
    } }
void save_results(FILE *fp, double C[], double time) {
    fprintf(fp, "# Time = %.2f s\n", time);
    fprintf(fp, "%-12s %-12s\n", "Position(m)", "Concentration");
    for(int i = 0; i < NX; i++) {
        double x = -L/2 + i * DX;
        fprintf(fp, "%10.2f %14.4f\n", x, C[i]);
    }
    fprintf(fp, "\n\n");  // Two blank lines for gnuplot index
}
```

RESULT:

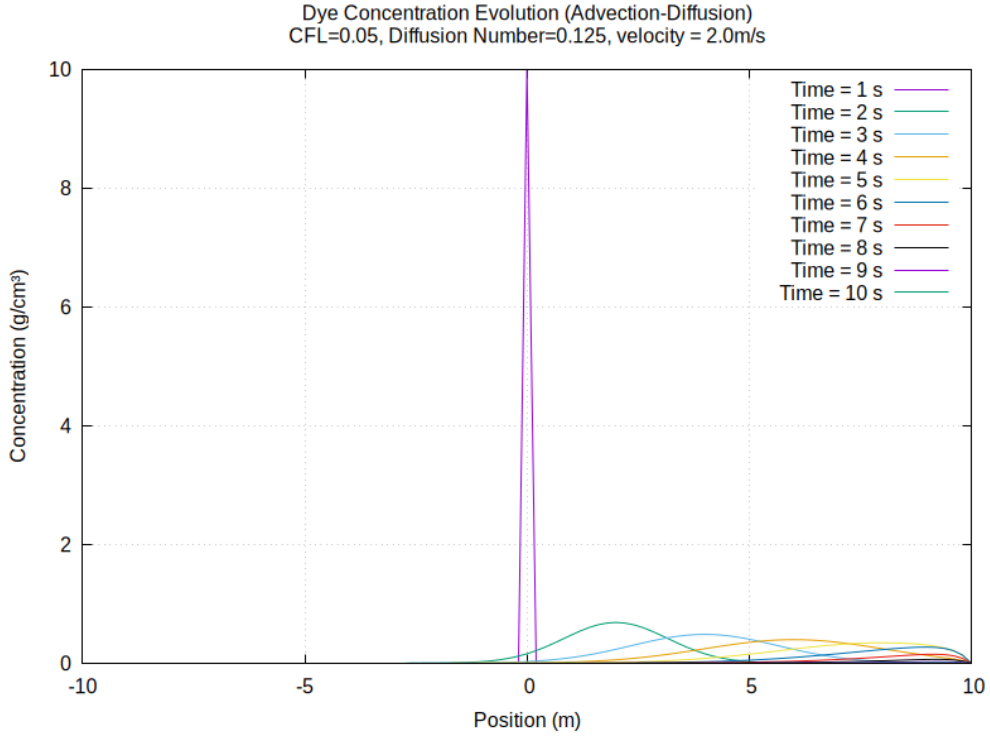
For the time step 0.01 and 0 m/s flow velocity:



For the time step 0.01 and 1 m/s flow velocity:



For the time step 0.01 and 2 m/s flow velocity:



CONCLUSION:

The analysis of the advection-diffusion process highlights the intricate balance between flow-driven transport and molecular diffusion in governing the behavior of dye dispersion. At higher flow velocities, advection dominates, rapidly displacing the dye downstream and reducing its concentration at fixed observation points due to shorter residence times. The dye's center of mass aligns with the mean flow velocity, as predicted by the advective component of the governing equation.

Conversely, diffusion drives lateral spreading, expanding the dye's spatial footprint proportionally to the diffusion coefficient and time, particularly evident in stagnant conditions where a Gaussian distribution emerges. When advection speeds are low, diffusion prevails, leading to uniform dispersion in all directions. The interplay between these mechanisms dictates the dye cloud's behavior. high velocities produce a narrow, fast-moving plume, while low velocities result in a broader, slower-spreading cloud. Over time, the maximum concentration diminishes as both advection and diffusion disperse the dye, with faster declines observed under stronger advection. These insights are critical for applications such as pollutant management and environmental modeling, where understanding the dual roles of advection and diffusion ensures accurate predictions of substance transport in natural and engineered systems.